

```

from flask import Flask, request, jsonify, send_from_directory
import requests
import os
import threading
import time
import json

app = Flask(__name__, static_folder='.', static_url_path='')

ANTHROPIC_API_KEY = os.environ.get('ANTHROPIC_API_KEY')
RENDER_URL = os.environ.get('RENDER_EXTERNAL_URL', 'https://ptrain.onrender.com')
SUPABASE_URL = 'https://ysztaecwyjuqchmcgxjr.supabase.co'
SUPABASE_KEY = 'sb_publishable_2Y6Azzpv7SNDVRMng3QU7Q_bhiWuV2i'
EO_API_KEY = 'eo_347e5fd056d045949689c03e7b6ab07ea37140c549188ffb56bded816c850462'
EO_LIST_ID = 'badf6dd4-6fa7-11f1-bb46-elbea4e2db03'

SUPABASE_HEADERS = {
    'apikey': SUPABASE_KEY,
    'Authorization': f'Bearer {SUPABASE_KEY}',
    'Content-Type': 'application/json',
    'Prefer': 'return=representation'
}

def cors(response):
    response.headers['Access-Control-Allow-Origin'] = '*'
    response.headers['Access-Control-Allow-Methods'] = 'GET, POST, OPTIONS'
    response.headers['Access-Control-Allow-Headers'] = 'Content-Type'
    return response

@app.route('/')
def home():
    return send_from_directory('.', 'ptrain.html')

@app.route('/manifest.json')
def manifest():
    return send_from_directory('.', 'manifest.json')

@app.route('/icon-192.png')
def icon192():
    return send_from_directory('.', 'icon-192.png')

@app.route('/icon-512.png')
def icon512():
    return send_from_directory('.', 'icon-512.png')

```

```

@app.route('/ping')
def ping():
    return jsonify({'status': 'ok'}), 200

def keep_alive():
    time.sleep(30)
    while True:
        try:
            requests.get(f'{RENDER_URL}/ping', timeout=10)
        except Exception:
            pass
        time.sleep(840)

# — SUPABASE USER STATE —————

@app.route('/api/state/load', methods=['POST', 'OPTIONS'])
def load_state():
    if request.method == 'OPTIONS':
        return cors(jsonify({'ok': True}))
    try:
        email = request.get_json().get('email', '').lower().strip()
        if not email:
            return cors(jsonify({'error': 'No email'})), 400
        r = requests.get(
            f'{SUPABASE_URL}/rest/v1/ptrain_users?email=eq.{email}&select=*',
            headers=SUPABASE_HEADERS, timeout=10
        )
        data = r.json()
        if data and len(data) > 0:
            return cors(jsonify({'found': True, 'state': data[0].get('state', {})}))
        return cors(jsonify({'found': False}))
    except Exception as e:
        return cors(jsonify({'error': str(e)})), 500

@app.route('/api/state/save', methods=['POST', 'OPTIONS'])
def save_state():
    if request.method == 'OPTIONS':
        return cors(jsonify({'ok': True}))
    try:
        body = request.get_json()
        email = body.get('email', '').lower().strip()
        if not email:
            return cors(jsonify({'error': 'No email'})), 400
        state = body.get('state', {})
        plan = body.get('plan')
        # Upsert - insert or update

```

```

        r = requests.post(
            f'{SUPABASE_URL}/rest/v1/ptrain_users',
            headers={**SUPABASE_HEADERS, 'Prefer': 'resolution=merge-duplicates',
            json={'email': email, 'state': state, 'plan': plan, 'updated_at': 'no
            timeout=10
        )
        return cors(jsonify({'ok': True, 'status': r.status_code}))
    except Exception as e:
        return cors(jsonify({'error': str(e)})), 500

@app.route('/api/state/upgrade', methods=['POST', 'OPTIONS'])
def upgrade_plan():
    if request.method == 'OPTIONS':
        return cors(jsonify({'ok': True}))
    try:
        body = request.get_json()
        email = body.get('email', '').lower().strip()
        plan_tier = body.get('plan', 'pro')
        if not email:
            return cors(jsonify({'error': 'No email'})), 400
        r = requests.post(
            f'{SUPABASE_URL}/rest/v1/ptrain_users',
            headers={**SUPABASE_HEADERS, 'Prefer': 'resolution=merge-duplicates',
            json={'email': email, 'state': {'userPlan': plan_tier, 'plansUsed': 0
            timeout=10
        )
        return cors(jsonify({'ok': True}))
    except Exception as e:
        return cors(jsonify({'error': str(e)})), 500

# — EMAIL OCTOPUS WELCOME —————

@app.route('/api/email/subscribe', methods=['POST', 'OPTIONS'])
def subscribe():
    if request.method == 'OPTIONS':
        return cors(jsonify({'ok': True}))
    try:
        body = request.get_json()
        email = body.get('email', '').lower().strip()
        name = body.get('name', '')
        if not email:
            return cors(jsonify({'error': 'No email'})), 400

        # Add to EmailOctopus list
        requests.post(
            f'https://emailoctopus.com/api/2.0/lists/{EO_LIST_ID}/contacts',
            json={

```

```

        'api_key': EO_API_KEY,
        'email_address': email,
        'fields': {'FirstName': name},
        'status': 'SUBSCRIBED'
    },
    timeout=10
)

```

```

# Send welcome email via EmailOctopus

```

```

welcome_body = f"""Hi {name or 'there'} 🙌

```

Welcome to PTrAIIn – your AI personal trainer built on real PT expertise.

Here's what you've just unlocked:

⚡ Your first complete AI training plan – fully personalised to your goals, equip

🥗 A full daily nutrition plan – macros, meals, calorie targets – built around yc

💬 Your AI Coach – ask anything, anytime. Training questions, nutrition advice, f

📈 Progressive overload tracking – every weight you lift is remembered. Next sess

To get started, head back to the app and tap Build My Plan:

<https://ptrain.onrender.com>

Built by a certified PT with 8 years at Gymbox – this isn't another generic AI to

Any questions? Just reply to this email.

Let's get to work 💪

Tel

PTrAIIn – Certified PT · 8 Years at Gymbox

"""

```

    # For now log it - full transactional email needs EO campaign setup

```

```

    # This structure is ready for when transactional emails are configured

```

```

    print(f'Welcome email queued for {email}')

```

```

    return cors(jsonify({'ok': True}))

```

```

except Exception as e:

```

```

    return cors(jsonify({'error': str(e)})), 500

```

```

# — AI GENERATION —

```

```

def build_prompt(profile):

```

```

    p = profile

```

```

        return f"""You are PTrAIIn, an AI personal trainer and nutrition coach built o

Profile: Name={p.get('name','User')}, Age={p.get('age','?')}, Weight={p.get('weig
Goal: {p.get('goal','General Fitness')} | Level: {p.get('level','Intermediate')}
Equipment: {p.get('equipment','Full Gym')} | Focus: {p.get('muscles','Full Body')}
Sleep: {p.get('sleep','7')}hrs/night | Stress: {p.get('stress','Moderate')} | Die
Injuries: {p.get('inj','None')} | Food allergies: {p.get('allergies','None')}

CRITICAL: Respond ONLY with valid complete JSON. Keep text brief. No markdown, no
{"planName":"string","coachMessage":"personal 1-sentence message to {p.get('name

@app.route('/api/generate', methods=['POST', 'OPTIONS'])
def generate():
    if request.method == 'OPTIONS':
        return cors(jsonify({'ok': True}))
    if not ANTHROPIC_API_KEY:
        return cors(jsonify({'error': {'message': 'ANTHROPIC_API_KEY not set'}}))
    try:
        body = request.get_json()
        if body.get('profile'):
            messages = [{'role': 'user', 'content': build_prompt(body['profile'])}
            system = None
        else:
            messages = body.get('messages', [])
            system = body.get('system')
        payload = {
            'model': body.get('model', 'claude-sonnet-4-6'),
            'max_tokens': body.get('max_tokens', 8000),
            'messages': messages,
        }
        if system:
            payload['system'] = system
        anthropic_response = requests.post(
            'https://api.anthropic.com/v1/messages',
            headers={
                'Content-Type': 'application/json',
                'x-api-key': ANTHROPIC_API_KEY,
                'anthropic-version': '2023-06-01',
            },
            json=payload,
            timeout=120,
        )
        return cors(jsonify(anthropic_response.json()))
    except Exception as e:
        return cors(jsonify({'error': {'message': str(e)}})), 500

if __name__ == '__main__':

```

```
t = threading.Thread(target=keep_alive, daemon=True)
t.start()
app.run(host='0.0.0.0', port=int(os.environ.get('PORT', 8080)))
```